

Frontend — Angular Application

▼ Relevant source files

Frontend/STUDER/PROJECT_DOCUMENTATION.md

Frontend/STUDER/angular.json

Frontend/STUDER/package-lock.json

Frontend/STUDER/package.json

Frontend/STUDER/public/favicon.ico

Frontend/STUDER/src/app/app.component.html

Frontend/STUDER/src/app/app.component.ts

Frontend/STUDER/src/app/app.config.ts

Frontend/STUDER/src/main.ts

The STUDER frontend is a Single Page Application (SPA) built with **Angular 19.2.0**

`package.json` 13 It is designed as a modular, reactive interface that consumes the Spring Boot REST API. The application leverages modern Angular features such as the new control flow syntax (`@if`, `@for`) and standalone components `PROJECT_DOCUMENTATION.md` 10

System Architecture Overview

The frontend follows a clean separation of concerns, dividing the codebase into core infrastructure, shared utilities, and domain-specific features.

Directory Structure

- `src/app/core` : Singleton services, HTTP interceptors, authentication state, and global configurations `PROJECT_DOCUMENTATION.md` 21
- `src/app/features` : Business modules (e.g., Discussions, Messaging, Notifications) `PROJECT_DOCUMENTATION.md` 22
- `src/app/shared` : Reusable UI components like the rich-text system and entity popovers `PROJECT_DOCUMENTATION.md` 28

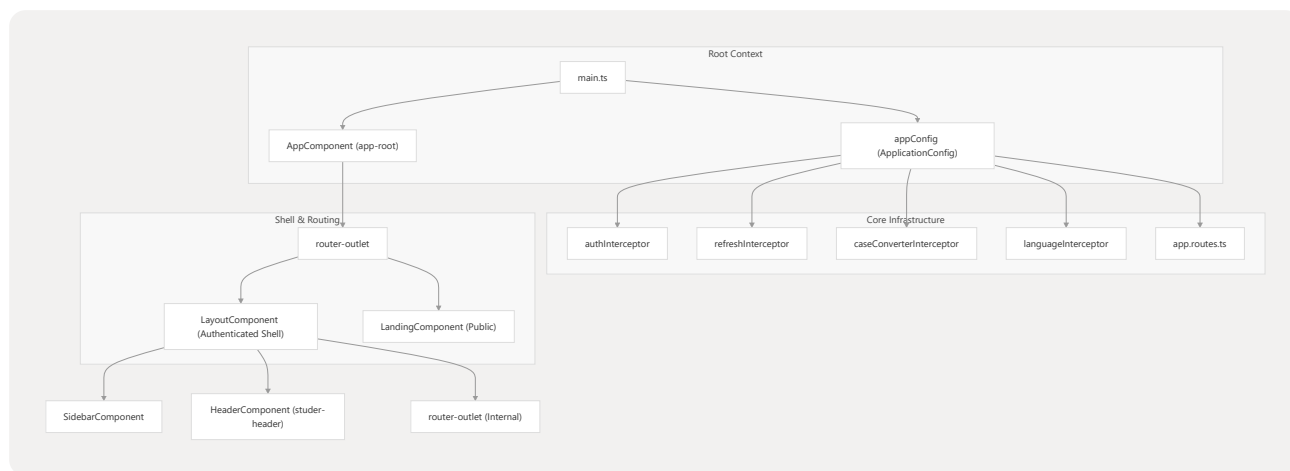
- `src/app/config` : Global constants and API base configurations

PROJECT_DOCUMENTATION.md 29

Component Hierarchy & Bootstrapping

The application is bootstrapped in `main.ts` using the `appConfig` provider `main.ts` 5 The `AppComponent` serves as the root shell, containing only a `<router-outlet>` to host the application's various layouts and views `app.component.html` 1

Component Relationship Diagram



Sources: `main.ts` 1-7 `app.config.ts` 1-18 `app.component.ts` 7-12

`app.component.html` 1 `PROJECT_DOCUMENTATION.md` 19-30

Core Configuration & Bootstrapping

The `appConfig` object in `src/app/app.config.ts` defines the global providers for the application. This includes zone detection, routing, and the HTTP client pipeline `app.config.ts` 11-18

HTTP Interceptor Pipeline

The application uses a functional interceptor chain to handle cross-cutting concerns for every API request:

1. `caseConverterInterceptor` : Automatically transforms request bodies to `snake_case` and response bodies to `camelCase` to bridge Java/Python naming conventions with TypeScript `app.config.ts` 8

- 2. `refreshInterceptor` : Detects 401 Unauthorized errors and attempts to rotate JWT tokens using the refresh token cookie `app.config.ts` 7
- 3. `authInterceptor` : Injects the `Authorization: Bearer <token>` header into outgoing requests `app.config.ts` 5
- 4. `languageInterceptor` : Attaches the `Accept-Language` header based on the user's current selection `app.config.ts` 6

For details, see [Core Services & HTTP Layer](#).

Global Shell (AppComponent)

The `AppComponent` acts as the entry point for global service initialization. Upon `ngOnInit`, it triggers the initialization of the `LanguageService` and `ThemeService` to ensure the UI matches user preferences immediately upon load `app.component.ts` 21-24

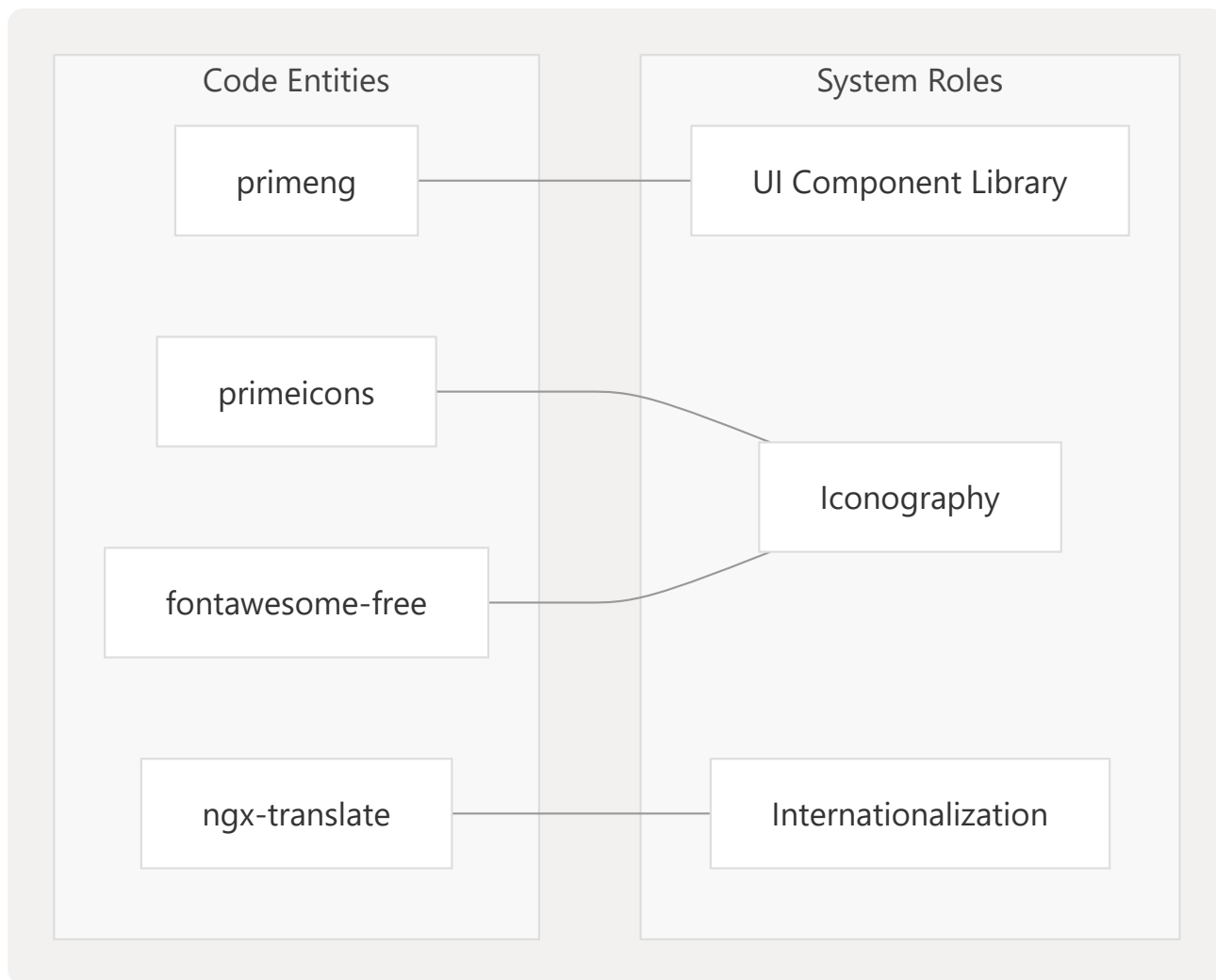
Feature	Service	Responsibility
Routing	<code>RouterOutlet</code>	Dynamic injection of feature components <code>app.component.html</code> 1
Theming	<code>ThemeService</code>	Manages light/dark mode via CSS variables <code>app.component.ts</code> 18-23
i18n	<code>LanguageService</code>	Sets the application locale (English/Spanish) <code>app.component.ts</code> 17-22

Sources: `app.component.ts` 13-25 `app.component.html` 1

Technical Stack & UI Components

The UI is built primarily using **PrimeNG**, supplemented by **Font Awesome** for iconography `PROJECT_DOCUMENTATION.md` 11-14

Technology Mapping



Sources: `package.json` 12-28 `PROJECT_DOCUMENTATION.md` 9-15

Subsystem Documentation

The following child pages provide detailed technical specifications for the major frontend subsystems:

- **Routing & Layout Shell**: Documentation of `app.routes.ts`, the `authGuard`, and the `LayoutComponent` which provides the sidebar and header structure.
- **Core Services & HTTP Layer**: Detailed breakdown of the interceptor logic, `AuthStateService` management, and API configuration.
- **Theming & Internationalization**: Implementation details of the `ThemeService` (CSS variables) and `LanguageService` (`ngx-translate`).
- **Header & Notification Panel**: Technical overview of the `HeaderComponent` search pipeline and the reactive `NotificationPanelComponent`.

- Rich Text System: Deep dive into the `RichTextParserService` regex tokenizer and the entity popover system for `@user` and `#tag` mentions.